

1. ОБЩЕЕ ПОНЯТИЕ ПРОЦЕССОРОВ С АРХИТЕКТУРОЙ ARM

1.1. Общее определение архитектуры ARM

Архитектура ARM (от англ. *Advanced RISC Machine* — усовершенствованная RISC-машина) — система команд и семейство описаний и готовых топологий 32-битных и 64-битных микропроцессорных ядер, разрабатываемых компанией ARM Limited.

Рисунок 1.1 – Процессор ARM

1.2. RISC-машина

RISC (англ. *reduced instruction set computer* — вычислитель с сокращенным набором команд) — архитектурный подход к проектированию процессоров, в которой быстродействие увеличивается за счёт такого кодирования инструкций, чтобы их декодирование было более простым, а время выполнения — меньшим. В системах команд первых RISC-процессоров даже отсутствовали команды умножения и деления. Это также облегчает повышение тактовой частоты и делает более эффективной суперскалярность (распараллеливание инструкций между несколькими исполнительными блоками).

1.3. Семейства процессоров архитектуры ARM

В наши дни выделяют несколько семейств процессоров ARM:

1) ARM7 (с тактовой частотой до 60-72 МГц), предназначенные для мобильных телефонов средней ценовой категории и встраиваемых решений средней производительности. В настоящее время активно вытесняется новым семейством Cortex.

2) ARM9, ARM11 (с частотами до 1 ГГц) для более мощных мобильных телефонов, карманных компьютеров и встраиваемых решений высокой производительности.

3) Cortex A — семейство процессоров пришедшее на смену ARM9 и ARM11.

4) Cortex M — семейство процессоров пришедшее на смену ARM7, также призванное занять новую для ARM нишу встраиваемых решений низкой производительности. В семействе присутствуют четыре значимых ядра:

1) Cortex-M0, Cortex-M0+ (более энергоэффективное) и Cortex-M1 (оптимизировано для применения в ПЛИС) с архитектурой ARMv6-M;

2) Cortex-M3 с архитектурой ARMv7-M;

3) Cortex-M4 (добавлены SIMD-инструкции, опционально FPU) и Cortex-M7 (FPU с поддержкой чисел одинарной и двойной точности) с архитектурой ARMv7E-M;

4) Cortex-M23 и Cortex-M33 с архитектурой ARMv8-M ARMv8-M.

1.4.Преимущества и недостатки

Преимущества процессора ARM:

1) Доступность создания - процессор ARM весьма доступный, так как для его создания не требуется дорогостоящее оборудование. По сравнению с другими процессорами, он создается по гораздо меньшей цене. Вот почему они подходят для изготовления недорогих мобильных телефонов и других электронных устройств.

2) Низкое энергопотребление - процессоры ARM имеют меньшее энергопотребление. Первоначально они были предназначены для работы на меньшей мощности. У них даже меньше транзисторов в их архитектуре. У них есть различные другие функции, которые позволяют это сделать.

3) Работает быстрее - ARM выполняет одну операцию за раз. Это ускоряет работу. Он имеет меньшую задержку, что ускоряет время отклика.

4) Функция многопроцессорной обработки - процессоры ARM разработаны таким образом, чтобы их можно было использовать в многопроцессорных системах, где для обработки информации используется более одного процессора. Первый процессор ARM, представленный под названием ARMv6K, имел возможность поддерживать 4 процессора вместе со своим оборудованием.

5) Лучшее время автономной работы - процессоры ARM имеют лучшее время автономной работы. Это видно из администрирования устройств, которые используют процессоры ARM, и тех, которые этого не делают. Те, которые использовали процессоры ARM, работали дольше и разряжались позже, чем те, которые не работали на процессорах ARM.

6) Простые схемы - процессоры ARM имеют простые схемы, поэтому они очень компактны и могут использоваться в устройствах меньшего размера (несколько устройств становятся меньше и компактнее из-за требований клиентов).

Недостатки процессора ARM:

1) Он не совместим с X86, поэтому его нельзя использовать в Windows.

2) Скорость ограничена в некоторых процессорах, что может создать проблемы.

3) Инструкции по планированию сложны в случае процессоров ARM.

4) Должно быть надлежащее выполнение инструкций программистом. Это связано с тем, что вся производительность процессоров ARM зависит от их выполнения.

5) Процессор ARM нуждается в очень высококвалифицированных программистах. Это связано с важностью и сложностью выполнения (процессор показывает меньшую производительность при неправильном выполнении).

2.ВСТРОЕННЫЕ СИСТЕМЫ ARM

2.1.Философия дизайна RISC

Ядро ARM использует архитектуру RISC. RISC — это философия дизайна, направленная на предоставление простых и мощных инструкций, которые выполняются в течение одного цикла на высокой тактовой частоте. Философия RISC концентрируется на уменьшении сложности инструкций, выполняемых аппаратным обеспечением, потому что легче обеспечить большую гибкость и интеллектуальность в программном обеспечении, чем в аппаратном. В результате дизайн RISC предъявляет более высокие требования к компилятору. Напротив, традиционный компьютер со сложным набором команд (CISC) больше полагается на аппаратное обеспечение для выполнения инструкций, и, следовательно, инструкции CISC более сложны.

Рисунок 2.1 – CISC против RISC.

Философия RISC реализуется с помощью трёх основных правил проектирования:

1) Инструкции. Процессоры RISC имеют уменьшенное количество классов инструкций. Эти классы предоставляют простые операции, каждая из которых может выполняться за один цикл. Компилятор или программист синтезирует сложные операции (например, операцию деления), комбинируя несколько простых инструкций. Каждая инструкция имеет фиксированную длину, что позволяет конвейеру извлекать будущие инструкции перед декодированием текущей инструкции. Напротив, в процессорах CISC инструкции часто имеют переменный размер и требуют много циклов для выполнения.

2) Конвейеры. Обработка инструкций разбита на более мелкие блоки, которые могут выполняться параллельно с помощью конвейеров. В идеале конвейер продвигается на один шаг в каждом цикле для максимальной пропускной способности. Инструкции могут быть декодированы на одном этапе конвейера. Нет необходимости в том, чтобы инструкция выполнялась минипрограммой, называемой микрокодом, как в процессорах CISC.

3) Регистры. RISC-машины имеют большой набор регистров общего назначения. Любой регистр может содержать либо данные, либо адрес. Регистры действуют как быстрое хранилище локальной памяти для всех

операций обработки данных. Напротив, процессоры CISC имеют специальные регистры для определенных целей.

Эти правила проектирования позволяют упростить RISC-процессор, и, таким образом, ядро может работать на более высоких тактовых частотах. Напротив, традиционные процессоры CISC более сложны и работают на более низких тактовых частотах. Однако в течение двух десятилетий различие между RISC и CISC стиралось, поскольку процессоры CISC реализовывали больше концепций RISC.

2.2.Философия дизайна ARM

Существует ряд физических особенностей, которые повлияли на конструкцию процессора ARM. Во-первых, портативные встраиваемые системы требуют некоторого питания от батареи. Процессор ARM был специально разработан таким образом, чтобы уменьшить энергопотребление и увеличить время работы от батареи, что необходимо мобильных телефонов и карманных персональных компьютеров (КПК).

Еще одним важным требованием является высокая плотность кода, поскольку встраиваемые системы имеют ограниченный объем памяти из-за ограничений по стоимости и/или физическому размеру. Высокая плотность кода полезна для приложений с ограниченной встроенной памятью, таких как мобильные телефоны и запоминающие устройства.

Кроме того, встроенные системы используют медленные и недорогие устройства памяти. **Для крупносерийных приложений, таких как цифровые камеры, при проектировании должен учитываться каждый цент.** Возможность использования недорогих запоминающих устройств дает существенную экономию.

ARM внедрила в процессор технологию аппаратной отладки, чтобы инженеры-программисты могли видеть, что происходит, пока процессор выполняет код. Благодаря большей наглядности инженеры-программисты могут быстрее решать проблемы, что напрямую влияет на время выхода продукта на рынок и снижает общие затраты на разработку.

Ядро ARM не является чистой архитектурой RISC из-за ограничений его основного приложения — встроенной системы. В некотором смысле сила ядра ARM заключается в том, что оно не заходит слишком далеко в концепции RISC. В современных системах ключом является не чистая скорость процессора, а общая эффективная производительность системы и энергопотребление.

2.3.Встроенное системное оборудование

Встроенные системы могут управлять многими различными устройствами, от небольших датчиков, установленных на производственной линии, до систем управления в реальном времени, используемых на космическом зонде НАСА. Все эти устройства используют комбинацию программных и аппаратных компонентов.

Каждый компонент выбран с точки зрения эффективности и, если применимо, предназначен для будущего расширения.

На рисунке 2.2 показано типичное встраиваемое устройство на базе ядра ARM. Каждая ячейка представляет собой свойство или функцию. Линии, соединяющие блоки, являются шинами, несущими данные.

Рисунок 2.2 – Пример встраиваемого устройства на базе ARM, микроконтроллера.

Мы можем разделить устройство на четыре основных аппаратных компонента:

1) Процессор ARM управляет встроенным устройством. Доступны различные версии процессора ARM, соответствующие желаемым рабочим характеристикам. Процессор ARM состоит из ядра (механизма выполнения, который обрабатывает инструкции и манипулирует данными) и окружающих компонентов, которые связывают его с шиной. Эти компоненты могут включать управление памятью и кэши.

2) Контроллеры координируют важные функциональные блоки системы. Двумя часто встречающимися контроллерами являются контроллеры прерываний и контроллеры памяти.

3) Периферийные устройства обеспечивают все внешние по отношению к чипу возможности ввода-вывода и отвечают за уникальность встроенного устройства.

4) Шина используется для связи между различными частями устройства.

3.ПРОГРАММИРОВАНИЕ

3.1.Команды обработки данных

В архитектуре ARM определен ряд команд *обработки данных* (в других архитектурах они часто называются логическими и арифметическими командами). Мы дадим их краткий обзор, потому что они необходимы для реализации конструкций более высокого уровня.

3.1.1Логические команды

К *логическим операциям* в ARM относятся команды AND (И), ORR (ИЛИ), EOR (ИСКЛЮЧАЮЩЕЕ ИЛИ) и BIC (сбросить бит). Это поразрядные операции над двумя операндами-источниками, результат которых записывается в регистр-приемник. Первым источником всегда является регистр, а вторым может быть другой регистр или непосредственный операнд. Еще одна логическая операция, MVN (MoVe and Not – перемещение и отрицание), определена как поразрядное НЕ, применяемое ко второму источнику (непосредственному операнду или регистру) с последующей записью в регистр-приемник. На рисунке 3.1 показано, как эти операции применяются к двум операндам-источникам 0x46A1F1B7 и 0xFFFF0000. Здесь же показано, какое значение оказывается в регистре-приемнике после выполнения команды.

Команда сброса бита (BIC) полезна для маскирования битов (сброса ненужных битов в 0). Команда BIC R6, R1, R2 вычисляет R1 AND NOT R2. Иными словами, BIC сбрасывает биты, поднятые в R2. В данном случае два старших байта R1 очищаются, или *маскируются*, а два незамаскированных младших байта R1, 0xF1B7, помещаются в R6. Замаскировать можно любое подмножество бит регистра.

Рисунок 3.1 – Логические операции

3.1.2. Команды сдвига

Команды сдвига сдвигают значение, находящееся в регистре, влево или вправо, при этом вышедшие за границу биты отбрасываются. В ARM имеются следующие команды сдвига: LSL (логический сдвиг влево), LSR (логический сдвиг вправо), ASR (арифметический сдвиг вправо) и ROR (циклический сдвиг вправо). Команды ROL не существует, потому что циклический сдвиг влево эквивалентен циклическому сдвигу вправо на дополнительное количество разрядов. На рисунке 3.2 показаны ассемблерный код и значения регистра-приемника для команд LSL, LSR, ASR и ROR при сдвиге на константное число бит. Значение в регистре R5 сдвигается, и результат помещается в регистр-приемник.

Рисунок 3.2 – Операции сдвига на величину, заданную непосредственно

3.1.3. Команды умножения

Умножение отличается от других арифметических операций тем, что при перемножении двух 32-битовых чисел получается 64-битовое число. В архитектуре ARM предусмотрены *команды умножения*, дающие 32- или 64-битовое произведение. Команда MUL перемножает два 32-битовых числа и дает 32-битовый результат. Команда MUL R1, R2, R3 перемножает значения в регистрах R2 и R3 и помещает младшие биты произведения в R1; старшие 32 бита отбрасываются. Эта команда полезна для умножения небольших чисел, произведение которых уместится в 32 бита. Команды UMULL (unsigned multiply long – длинное умножение без знака) и SMULL (signed multiply long – длинное умножение со знаком) перемножают два 32-битовых числа и порождают 64-битовое произведение. Например, UMULL R1, R2, R3, R4 вычисляет произведение значений в регистрах R3 и R4, рассматриваемых как целые без знака. Младшие 32 бита произведения помещаются в регистр R1, а старшие 32 бита – в регистр R2. У каждой из этих команд имеется также вариант с накоплением, MLA, SMLAL и UMLAL, который прибавляет произведение к накопительной сумме, 32- или 64-битовой. Эти команды повышают производительность в некоторых математических расчетах, например при

умножении матриц или обработке сигналов, когда требуется многократно выполнять операции умножения и сложения.

3.2.Эволюция архитектуры ARM

Процессор ARM1 был разработан британской компанией Acorn Computer для компьютеров BBC Micro в 1985 году как модернизация микропроцессора 6502, которые в то время широко использовались во многих персональных компьютерах. Не прошло и года, как за ним последовал процессор ARM2, который промышленно изготавливался для компьютера Acorn Archimedes. В изделии была реализована версия 2 набора команд ARM (ARMv2). Адресная шина была 26-битовой, а старшие 6 бит 32-битового счетчика команд использовались для хранения битов состояния. Вскоре адресная шина была расширена до полных 32 бит, а биты состояния переместились в специальный регистр текущего состояния программы (CPSR). В версии ARMv4, появившейся в 1993 году, были добавлены команды загрузки и сохранения полуслова, а также команды загрузки полуслов со знаком и без знака и байтов. Набор команд ARM подвергался многочисленным усовершенствованиям, которые описываются ниже. Оказавшийся чрезвычайно успешным процессор ARM7TDMI, выпущенный в 1995 году, привнес 16-битовый набор команд Thumb в виде версии ARMv4T с целью повысить плотность кода. В версии ARMv5TE были добавлены команды для цифровой обработки сигналов (digital signal processing, DSP) и необязательные команды для чисел с плавающей точкой. В версии ARMv6 добавились мультимедийные команды и был расширен набор команд Thumb.

В версии ARMv7 были усовершенствованы команды для работы с плавающей точкой и мультимедиа, которые получили общее название Advanced SIMD. Версия ARMv8 ознаменовалась совершенно новой 64-битовой архитектурой. По мере эволюции архитектуры добавлялись и другие команды для программирования системы.

3.2.1.Набор команд Thumb

Команды Thumb 16-битовые, что позволяет увеличить плотность кода; они повторяют обычные команды ARM, но с рядом ограничений:

- 1) Могут обращаться только к восьми младшим регистрам.
- 2) Один и тот же регистр играет роль источника и приемника.
- 3) Поддерживаются более короткие непосредственные операнды.
- 4) Отсутствует условное выполнение.
- 5) Всегда устанавливаются флаги состояния.

Почти у всех команд ARM есть эквивалентные команды Thumb. Поскольку команды Thumb скованы ограничениями, для написания эквивалентной программы их понадобится больше. Однако длина этих команд в два раза

меньше, поэтому общий размер написанного с их помощью кода составляет примерно 65% от эквивалентного кода с использованием полной системы команд ARM. Набор команд Thumb полезен не только для уменьшения размера кода и стоимости требуемой для его хранения памяти. Он еще позволяет использовать дешевую 16-битовую шину для памяти команд и сокращает энергопотребление при выборке команд из памяти. Кодирование команд в режиме Thumb сложнее и не так единообразно, как в режиме ARM, поскольку требуется упаковать как можно больше полезной информации в 16-битовое полуслово. На рисунке 3.3 показано, как кодируются некоторые часто встречающиеся команды Thumb. В старших битах задается тип команды. В командах обработки данных обычно указываются два регистра, один из которых играет роль источника и приемника. Любая команда всегда устанавливает флаги состояния. В командах сложения, вычитания и сдвига можно задавать короткий непосредственный операнд. В командах условного перехода задается 4-битовый код условия и короткое смещение, а в командах безусловного перехода допустимо более длинное смещение. Отметим, что команда BX принимает 4-битовый идентификатор регистра, поэтому может получить доступ к регистру связи LR. Существуют специальные разновидности команд LDR, STR, ADD и SUB, работающие относительно указателя стека SP (чтобы можно было получить доступ к кадру стека при обработке вызова функции). Еще одна разновидность команды LDR загружает данные относительно счетчика команд PC (для доступа к пулу литералов). Варианты команд ADD и MOV позволяют обращаться ко всем 16 регистрам. В команде BL необходимо всегда задавать два полуслова, определяющих 22-битовый конечный адрес.

Впоследствии набор команд Thumb был усовершенствован. В набор Thumb-2 добавили ряд 32-битовых команд для повышения производительности типичных операций и для того, чтобы в режиме Thumb можно было написать любую программу. Команда принадлежит набору Thumb-2, если 5 старших бит принимают одно из значений: 11101, 11110, 11111. В этом случае процессор выбирает из памяти второе полуслово, содержащее конец команды. Процессоры серии Cortex-M работают исключительно в режиме Thumb.

Рисунок 3.3 – Примеры кодирования команд из набора Thumb

4.ПРОЦЕССОРЫ ARM В КОМПЬЮТЕРАХ APPLE

4.1.ПЕРЕХОД MAC НА APPLE SILICON

Переход Mac на Apple silicon — запланированный двухлетний процесс внедрения Apple silicon на основе ARM64 и отказа от Intel x86-64 в линейке компьютеров Apple Macintosh. О переходе компания объявила на Всемирной конференции для разработчиков (WWDC). 10 ноября 2020 года Apple

анонсировала Apple M1(изображён на рисунке 4.1), свою первую систему-на-чипе на базе архитектуры ARM для использования в компьютерах Mac, а также обновленные модели Mac Mini, MacBook Air и 13-дюймового MacBook Pro на его основе. Ниже, на рисунке 4.2 изображено сравнение материнских плат компьютеров iMac предыдущего и нового поколений соответственно.

Рисунок 4.1 – Центральный процессор Apple M1

Рисунок 4.2 – Сверху - материнская плата предыдущего поколения с процессором Intel (x86),
снизу плата 2021 года с чипом M1 (ARM)

4.1.1.Архитектура Apple M1

Apple M1 — серия систем на кристалле ARM-архитектуры компании Apple из серии Apple silicon, используемая в компьютерах Mac, ноутбуках MacBook и планшетах iPad Pro и iPad Air, производится контрактным производителем TSMC на 5-нанометровом техпроцессе и содержит около 16 миллиардов транзисторов.

Apple M1 имеет четыре высокопроизводительных ядра «Firestorm» и четыре ядра низкого энергопотребления «Icestorm», что обеспечивает конфигурацию, аналогичную ARM big.LITTLE и процессорам Lakefield от Intel. Это сочетание позволяет оптимизировать энергопотребление; эта возможность отсутствует в устройствах архитектуры Apple-Intel. Apple утверждает, что ядра низкого энергопотребления используют одну десятую мощности высокопроизводительных. Высокопроизводительные ядра имеют 192 КБ кэша команд и 128 КБ кэша данных и совместно используют 12 МБ кэша L2. Аналогичны характеристики ядер низкого энергопотребления таковы: 128 КБ кэша команд, 64 КБ кэша данных и общий 4 МБ кэша L2. Icestorm «E cluster» имеет частоту 0,6-2,064 ГГц и максимальную потребляемую мощность 1,3 Вт, Firestorm «P cluster» — частоту 0,6-3,204 ГГц и максимальную потребляемую мощность 13,8 Вт.

4.2.Сравнение ARM и x86

Большинство мобильных устройств, iPhone и Android работают на ARM. Qualcomm, HUAWEI Kirin, Samsung Exynos и Apple A13/A14 Bionic — это все ARM-процессоры. А в компьютерах доминирует x86 от компаний Intel и AMD. x86 — так называется по последним цифрам семейства классических процессоров Intel 70-80х годов, изображён на рисунке 4.3.

Рисунок 4.3 – Пример классического процессора Intel 70-80х годов

4.2.1.Ключевые отличия

Между ARM и x86 есть два ключевых отличия.

Первое — это набор инструкций. x86 процессоры используют сложный набор инструкций - CISC - Complex Instruction Set Computing. ARM процессоры наоборот используют упрощенный набор инструкций — RISC - Reduced

Instruction Set Computing. Набор инструкций RISC подробно рассмотрен в подразделе 2.1.

Второе отличие — это микроархитектура.

Arm основан на RISC (Reduced Instruction Set Computing), в то время как Intel (x86) - CISC (Complex Instruction Set Computing). Инструкции процессора Arm достаточно атомарны, с очень тесной корреляцией между количеством инструкций и микрооперациями. CISC, для сравнения, предлагает гораздо больше инструкций, многие из которых выполняют несколько операций (например, оптимизированная математика и перемещение данных). Это приводит к лучшей производительности, но большему энергопотреблению, расшифровыванию этих сложных инструкций. Тем не менее, границы между RISC и CISC в наши дни немного размыты, при этом каждая из них заимствует идеи друг у друга и широкий спектр ядер процессора, построенных на вариациях архитектуры. Кроме того, возможность настройки архитектуры Arm означает, что партнеры, такие как Apple, могут добавлять свои собственные более сложные инструкции.

Ниже приведён пример кода на языке Ассемблера одной и той же операции для x86 и ARM.

Пример кода на языке Ассемблера для x86:

- 1) MOV AX, 15; AH = 00, AL = 0Fh;
- 2) AAA; AH = 01, AL = 05;
- 3) RET.

Пример кода на языке Ассемблера для ARM:

- 1) MOV R3, #10;
- 2) AND R2, R0, #0xF;
- 3) CMP R2, R3;
- 4) IT LT;
- 5) BLT elsebranch;
- 6) ADD R2, #6;
- 7) ADD R1, #1;
- 8) elsebranch.;
- 9) END.

Архитектура RISC проще и удобнее в использовании согласно следующим преимуществам:

- 1) проще работа с памятью;
- 2) более богатая регистровая архитектура;
- 3) легче делать 32/64/128 разряды;
- 4) легче оптимизировать;
- 5) меньше энергопотребление;
- 6) проще масштабировать и делать отладку.

Для примера рассмотрим два процессора одного поколения. ARM1 и Intel 386. При схожей производительности ARM вдвое меньше по площади. А транзисторов на нем в 10 раз меньше: 25 тысяч против 275 тысяч. Энергопотребление тоже отличается на порядок: 0.1 Ватт против 2 Ватт у Intel.

Поэтому, к примеру смартфоны, которые работают на ARM процессорах с архитектурой RISC, долго живут, не требуют активного охлаждения и крайне быстрые.

4.2.2.Итоги сравнения

За последнее десятилетие соперничества Arm против x86 Arm выиграл как выбор для устройств с низким энергопотреблением, таких как смартфоны. Архитектура также делает успехи в ноутбуках, что мы собственно видим на примере сотрудничества ARM и Apple, и других устройствах, где востребована повышенная энергоэффективность.

Тем не менее, Arm и x86 по-прежнему явно отличаются с инженерной точки зрения, и у них по-прежнему есть индивидуальные сильные и слабые стороны. Тем не менее, потребительские сценарии использования в двух странах становятся размытыми, поскольку экосистемы все чаще поддерживают обе архитектуры. Тем не менее, хотя в сравнении Arm и x86 есть кроссовер, именно Arm, несомненно, останется предпочтительной архитектурой для индустрии смартфонов в обозримом будущем. Архитектура также демонстрирует большие перспективы для вычислений и эффективности класса ноутбуков.

ЗАКЛЮЧЕНИЕ

В последние годы языки программирования эволюционировали, чтобы обеспечить поддержку многим архитектурам, и в большинстве случаев практически не требуется никаких усилий для поддержки таких архитектур, как Arm.

Проекты веб-уровня являются отличной отправной точкой для интеграции Arm. Например, такие проекты, как **Apache** и **NGINX**, являются очень популярными веб-серверами, которые уже поддерживают архитектуру Arm. Стоит отметить, что поддержка Arm здесь не единственное преимущество, так как запуск NGINX в качестве кэш-сервера или обратного прокси-сервера в среде ARM позволит повысить производительность по сравнению с x86.

Базы данных в памяти - это еще одна область, где архитектура Arm может обеспечить улучшения. Используя архитектуру RISC, такие приложения, как Redis, могут достичь привлекательной производительности по сравнению со средой x86.

Архитектура Arm кажется отличной альтернативой системам x86, и из-за своей эффективности компании инвестируют в свое будущее для облачных вычислений. Трансформация приложений и кластеров может обеспечить экономию средств без ущерба для производительности.

Apache HTTP-сервер — свободный веб-сервер. Apache является кроссплатформенным ПО, поддерживает операционные системы Linux, BSD, Mac OS, Microsoft Windows, Novell NetWare, BeOS. Основными достоинствами Apache считаются надёжность и гибкость конфигурации. Nginx — веб-сервер и почтовый прокси-сервер, работающий на Unix-подобных операционных системах. Начиная с версии 0.7.52 появилась

экспериментальная бинарная сборка под Microsoft Windows. Игорь Сысоев начал разработку в 2002 году. Осенью 2004 года вышел первый публично доступный релиз.

Redis — резидентная система управления базами данных класса NoSQL с открытым исходным кодом, работающая со структурами данных типа «ключ — значение». Используется как для баз данных, так и для реализации кэшей, брокеров сообщений. Ориентирована на достижение максимальной производительности на атомарных операциях